

Phase 5: Google OAuth & JWT Authentication - Interview Prep Guide

Project Context

Objective: Implement user-specific history by adding Google OAuth authentication with JWT tokens, ensuring each user only sees their own generated documentation.

Problem Solved: Phases 1-4 had shared history across all users. Phase 5 isolated data by user.

Core Concepts

1. OAuth 2.0 (Google Sign-In)

What is OAuth 2.0?

- Open authentication standard allowing users to sign in via third-party providers
- User authenticates with Google, not your app
- Your app receives a token proving the user's identity
- More secure than storing passwords

Why Google OAuth?

- Users don't share passwords with your app
- Reduces your liability for credential management
- Single sign-on convenience
- Access to user profile data (name, email, picture)

Flow:

1. User clicks "Sign in with Google"
2. Frontend sends to Google's OAuth server
3. User authenticates with Google
4. Google returns ID token
5. Frontend sends token to backend
6. Backend verifies token with Google
7. Backend creates user if new, returns JWT
8. Frontend stores JWT in localStorage
9. All future API requests include JWT

Interview Q: "Explain the difference between OAuth and basic auth."

Answer: "OAuth is a delegation protocol. The user authenticates with a trusted provider (Google), who vouches for their identity. Your app never sees the password. Basic auth requires users to give you their credentials directly, which is less secure. OAuth is better for third-party integrations and user privacy."

2. JWT (JSON Web Tokens)

What is JWT?

- Stateless authentication token with three parts: Header.Payload.Signature
- Encoded (not encrypted) - you can read it but can't forge it without the secret

Structure:



- **Header:** Algorithm (HS256) and token type (JWT)
- **Payload:** User data (userId, expiration)
- **Signature:** HMAC-SHA256(header.payload, secret)

Why JWT over Session Tokens?

Feature	Session	JWT
Storage	Server database	Client (localStorage)
Scalability	Single server only	Distributed/microservices
Bandwidth	Requires DB lookup	Self-contained
State	Stateful	Stateless

Interview Q: "What's in a JWT and why is it secure?"

Answer: "JWT has three parts separated by dots. The signature is created using HMAC with a server secret. Even if someone reads the payload (it's base64, not encrypted), they can't modify it without the secret. The signature proves the token hasn't been tampered with. The server verifies the signature by recalculating it with the secret."

3. Token Expiration & Refresh Strategy

Access Token: Short-lived (7 days), used for API requests **Refresh Token:** Long-lived (30 days), used to get new access tokens

Simple Strategy (This Project):

- Single 7-day access token
- No refresh tokens (user logs in again after 7 days)
- Good for: Development, personal projects

Production Strategy:

- 15-min access token
- 30-day refresh token
- Refresh tokens stored in HttpOnly cookies

- Access tokens in Authorization header

Interview Q: "Why use short-lived access tokens?"

Answer: "If a token is compromised, the attacker only has 15 minutes of access instead of forever. After 15 minutes, they need the refresh token to get a new access token. Refresh tokens are stored in HttpOnly cookies (JavaScript can't access them), making them harder to steal."

Implementation Details

Backend Setup

1. Environment Variables:

```
env
```

```
GOOGLE_CLIENT_ID=123456789-abcdefg.apps.googleusercontent.com
JWT_SECRET=random_string_32_characters_long_very_secure
```

Why randomize JWT_SECRET?

- If compromised, attacker can forge tokens
- Rotate on security breach
- Different value per environment

2. User Model (MongoDB):

```
javascript
```

```
{
  googleId: String (unique),    // Google's user ID
  email: String,                // User's email
  name: String,                 // User's name
  picture: String,              // Profile picture URL
  createdAt: Date               // When user first signed up
}
```

Why googleId, not password?

- We never store passwords (Google handles authentication)
- googleId is unique and permanent
- Allows account linking later (same Google account)

3. Auth Middleware:

```
javascript
```

```
const verifyToken = (req, res, next) => {
  const token = req.headers.authorization.split(' ')[1];
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.userId = decoded.userId;
  next();
};
```

Middleware Pattern:

- Extract token from "Authorization: Bearer <token>"
- Verify signature using JWT_SECRET
- If valid, set req.userId for downstream routes
- If invalid, throw 401 error

Interview Q: "What happens if JWT_SECRET is leaked?"

Answer: "An attacker can forge tokens with any userId. All existing tokens become invalid. You must rotate the secret immediately, which logs out all users. In production, use refresh token rotation to minimize impact."

4. Google Login Route:

```
javascript

POST /api/auth/google-login
Body: { googleToken: "eyJ..." }

Response:
{
  token: "eyJ...", // JWT access token
  user: {
    id: "userId",
    name: "Siddharth",
    email: "sidsingh@gmail.com"
  }
}
```

Verification Flow:

1. Receive Google token (ID token from frontend)
2. Call `client.verifyIdToken()` with Google's public key
3. Extract user info (googleId, email, name)
4. Find or create User in database
5. Sign JWT with userId
6. Return token + user data

Why verify with Google?

- Ensures token came from Google (not forged)
- Ensures token hasn't expired
- Ensures token matches your Client ID

Frontend Setup

1. useContext (React Context API):

```
javascript

{
  user: { id, name, email },
  token: "eyJ...",
  loading: false,
  login: (userData, token) => {},
  logout: () => {}
}
```

Why Context?

- Makes user/token available everywhere without prop drilling
- Persists across page refreshes (via localStorage)
- Centralizes auth state

2. Login Flow:

```
User clicks "Sign in with Google"
  ↓
GoogleLogin component (from @react-oauth/google)
  ↓
Google opens popup for authentication
  ↓
User signs in with Google
  ↓
Google returns credentialResponse.credential (ID token)
  ↓
Send to backend: POST /api/auth/google-login
  ↓
Backend verifies & returns JWT + user data
  ↓
Frontend stores JWT in localStorage
  ↓
Redirect to "/" (protected route)
```

3. Axios Interceptor (JWT in Headers):

```
javascript
```

```
const axiosInstance = useMemo(() =>
  axios.create({
    baseURL: API_URL,
    headers: {
      Authorization: `Bearer ${token}`
    }
  }),
  [token]
);
```

Every API request now includes:

```
Authorization: Bearer eyJ...
```

Backend extracts & verifies:

javascript

```
const token = req.headers.authorization.split(' ')[1];
const decoded = jwt.verify(token, JWT_SECRET);
req.userId = decoded.userId;
```

4. Protected Routes:

javascript

```
const ProtectedRoute = ({ children }) => {
  const { token, loading } = useContext(UserContext);

  if (loading) return <div>Loading...</div>;
  if (!token) return <Navigate to="/login" />;

  return children;
};
```

Interview Q: "How do you prevent unauthenticated access?"

Answer: "Wrap routes in ProtectedRoute component. On mount, check if JWT exists in localStorage. If not, redirect to login. If loading, show loading state. This happens before sensitive data loads."

Security Best Practices

What We Did Right:

✓ JWT signed with secret (can't be forged)
 ✓ Token includes expiration
 ✓ Token stored in localStorage (survives page refresh)
 ✓ Backend verifies token on every request
 ✓ User data filtered by userId (can't see other users' docs)
 ✓ Google handles password security (we don't store it)

What Production Would Add:

✗ HttpOnly cookies instead of localStorage (prevents XSS token theft) ✗ HTTPS only (tokens sent over encrypted connection) ✗ Token rotation (short access + long refresh) ✗ Rate limiting on login endpoint ✗ CSRF protection on state parameter ✗ Logout token blacklist (prevent reuse after logout)

Interview Q: "Why use HttpOnly cookies instead of localStorage?"

Answer: "localStorage is vulnerable to XSS attacks - malicious JavaScript can read the token. HttpOnly cookies can't be accessed by JavaScript, only sent by the browser in requests. If someone injects malicious JS, they can't steal the token. Trade-off: CSRF attacks are slightly easier to perform but can be mitigated with CSRF tokens."

Common Issues & Solutions

Issue 1: origin_mismatch on Production

Cause: Production URL not in Google OAuth authorized origins **Solution:** Add

`https://your-domain.com` to Google Console → Credentials → Authorized Origins

Issue 2: CORS Errors

Cause: Frontend origin not in backend allowedOrigins **Solution:**

```
javascript

const allowedOrigins = [
  'http://localhost:3000',
  'https://your-frontend.vercel.app'
];
app.use(cors({ origin: allowedOrigins }));
```

Issue 3: Token Undefined in API Calls

Cause: axiosInstance created before token loaded **Solution:** Wrap in useMemo with [token] dependency

Issue 4: 401 Unauthorized on Protected Routes

Cause: Token expired or verifyToken middleware missing **Solution:** Check JWT_SECRET matches, check token format ("Bearer <token>")

Interview Q: "How would you debug a 401 error?"

Answer: "First, verify the token exists in localStorage. Check the Authorization header in Network tab to ensure format is 'Bearer token'. Verify JWT_SECRET on backend matches. Decode the token using jwt.io to check expiration. Check verifyToken middleware is applied to the route. If token is expired, delete localStorage and re-login."

Data Flow Diagram

FRONTEND

1. User clicks "Sign in with Google"
2. GoogleLogin component → Google OAuth popup
3. User authenticates with Google
4. Google returns ID token
5. POST /api/auth/google-login with ID token



BACKEND

6. Receive ID token
7. Verify with Google (client.verifyIdToken)
8. Extract: googleId, email, name, picture
9. Find or create User in MongoDB
10. Sign JWT with userId & JWT_SECRET
11. Return { token, user }



FRONTEND

12. Store JWT in localStorage
13. Store user in localStorage
14. Update UserContext (user, token)
15. Redirect to "/" (ProtectedRoute allows access)
16. Create axiosInstance with Authorization header
17. All API calls include "Authorization: Bearer token"



PROTECTED ROUTES (GET /api/docs, POST /api/generate-docs, etc)

18. Middleware: Extract token, verify signature
19. Set req.userId = decoded.userId
20. Query: Documentation.find({ userId: req.userId })
21. Return only user's docs (not other users')

Interview Q&A

Q1: "Explain how you implemented user authentication."

A: "I used Google OAuth 2.0 for login and JWT for stateless session management. When users sign in, they authenticate with Google, which returns an ID token. I send this to the

backend, which verifies it with Google's public key. After verification, I create a JWT containing the `userId` and sign it with a server secret. The frontend stores this JWT and includes it in every API request. The backend verifies the JWT signature and extracts the `userId` to filter data - each user only sees their own history."

Q2: "Why did you choose JWT over session cookies?"

A: "JWT is stateless - the server doesn't need to store session data. This makes it ideal for distributed systems and serverless functions. The token is self-contained, so any server can verify it without database lookups. Sessions would require a shared database or cache across Railway instances. For this project, JWT provides simplicity and scalability. In production, I'd use JWTs with refresh tokens and `HttpOnly` cookies for better security."

Q3: "How do you prevent users from seeing other users' data?"

A: "Every protected route uses `verifyToken` middleware to extract `userId` from the JWT. Then I filter queries by `userId`. For example, instead of `Documentation.find()`, I use `Documentation.find({ userId: req.userId })`. This ensures MongoDB only returns documents created by that user. If someone tries to forge a token or modify the `userId`, the signature verification fails."

Q4: "What security considerations did you make?"

A: "I store `JWT_SECRET` in environment variables (never committed). Tokens expire in 7 days to limit damage if compromised. I verify signatures on every request. I use HTTPS in production (enforced by Vercel and Railway). I filter data by `userId` server-side, never trusting client claims. I added CORS to only allow my frontend domain. For production, I'd add `HttpOnly` cookies, rate limiting, and token rotation."

Q5: "What happens if the JWT expires?"

A: "The backend's `jwt.verify()` throws an error, which the middleware catches and returns 401 Unauthorized. The frontend detects the 401, clears `localStorage`, and redirects to `/login`. The user must sign in again. In production, I'd use refresh tokens - the client stores a long-lived refresh token in `HttpOnly` cookies and uses the 15-minute access token for APIs. When the access token expires, the client uses the refresh token to get a new one without re-logging in."

Q6: "How do you handle CORS with authentication?"

A: "I added allowed frontend URLs to the backend's CORS configuration. But CORS only protects against browser requests - it checks the `Origin` header. For authentication, I also use JWT verification. Even if someone bypasses CORS, the backend still requires a valid JWT. The combination is defense in depth: CORS prevents unauthorized browser requests, JWT prevents token forgery."

Q7: "What would you do differently in production?"

A: "I'd use HttpOnly cookies for tokens instead of localStorage to prevent XSS. I'd implement refresh token rotation - access tokens last 15 min, refresh tokens last 30 days. I'd add rate limiting on login (10 attempts per minute per IP). I'd use HTTPS everywhere and add Content Security Policy headers. I'd log authentication events for audit trails. I'd implement token blacklisting on logout. I'd use more granular error messages (not 'Invalid token', just '401')."

Key Takeaways for Interviews

1. **OAuth 2.0** = Delegation of auth to trusted provider (Google)
 2. **JWT** = Stateless, self-contained token with signature
 3. **Verification** = Backend verifies JWT signature on every request
 4. **User Isolation** = Filter queries by userId, never trust client
 5. **Security** = Environment variables, HTTPS, token expiration, signature validation
 6. **Scalability** = Stateless JWT works across distributed systems
 7. **Production Upgrades** = HttpOnly cookies, refresh tokens, rate limiting
-

Related Topics to Study

- OAuth 2.0 specification (RFC 6749)
 - JWT best practices (RFC 7519)
 - OWASP authentication checklists
 - Refresh token patterns
 - PKCE (Proof Key for Code Exchange) for mobile apps
 - OpenID Connect (extends OAuth for authentication)
 - Session fixation attacks
 - Token storage security (localStorage vs cookies)
-

Resources

- [Google OAuth 2.0 Docs](#)
 - [JWT.io](#) - Decode and debug tokens
 - [auth0.com blog](#) - Auth best practices
 - [OWASP Authentication Cheat Sheet](#)
-

You can confidently explain OAuth flow, JWT structure, security considerations, and production improvements. Practice the Q&A section aloud before interviews!